AWS Database Blog

Create self-managed replicas for an Amazon RDS for Db2 instance for read scaling and disaster recovery

by Javeed Mohammed, Karthik Gopalakrishnan, Dean Capps, Feng Cai, and Ahmed Darwish | on 10 SEP 2024 | in Advanced (300), AWS Database Migration Service, RDS for Db2, Technical How-to | Permalink | Comments | Share

Resources

Amazon Aurora
Amazon DocumentDB
Amazon DynamoDB
Amazon ElastiCache
Amazon Keyspaces (for Apache Cassandra)
Amazon MemoryDB
Amazon Neptune
Amazon Neptune Analytics
Amazon RDS
Amazon Timestream
AWS Database Migration Service

Amazon Relational Database Service (Amazon RDS) for Db2 makes it simple to set up, operate, and scale IBM Db2 deployments in the cloud. IBM Db2 is a relational database that supports mission critical enterprise level and analytical workloads. Amazon RDS manages time-consuming database administration tasks, such as provisioning, patching, backups, recovery, failure detection, and recovery, so you can focus on applications and running your business.

In this post, we walk you through the steps to create self-managed replicas of your RDS for Db2 instance for read scaling and disaster recovery.

In a production environment, safeguarding against unforeseen disruptions is crucial for providing business continuity. Although Amazon RDS offers a resilient Multi-AZ configuration, it can't defend against every potential risk, such as natural disasters, malicious events, database corruption, or events that cause a workload to operate in a degraded state. Running out of another Region could be a mitigation strategy. To maintain uninterrupted business operations, it's essential to devise and validate a comprehensive disaster recovery (DR) plan.

Replicas serve a vital role in scalability by significantly enhancing performance and resilience for RDS for Db2 DB instances. They facilitate elastic scaling beyond the limitations of a single DB instance, which is particularly beneficial for read-intensive workloads. By creating one or more replicas of your primary RDS for Db2 instance, you can efficiently handle high-volume read requests across multiple data copies, thereby boosting aggregate read throughput. Additionally, these replicas can be swiftly promoted to independent DB instances as needed.

Solution overview

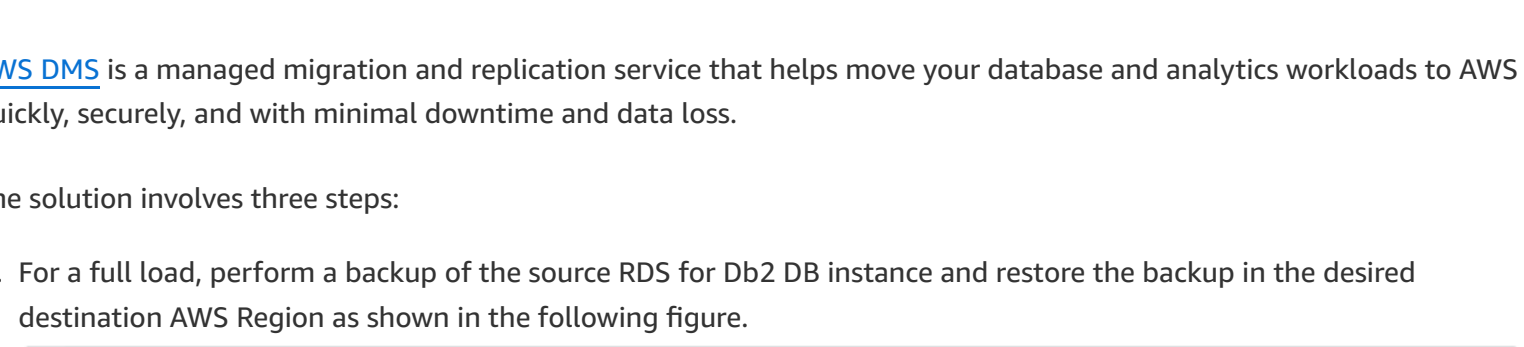
In this post, we explain how to use RDS Db2 Snapshot and AWS Database Migration Service (AWS DMS) to create cross Region replicas for your RDS for Db2 DB instance. If you want to use this replica for read scaling, there needs to be logic built at the application layer to direct only read traffic to the replica.

RDS Db2 Snapshot is a manual snapshot for Amazon RDS for Db2 is a user-initiated backup of your Db2 database instance at a specific point in time. It captures the entire database, including the data and transaction logs, allowing you to restore the database to that particular state if needed.

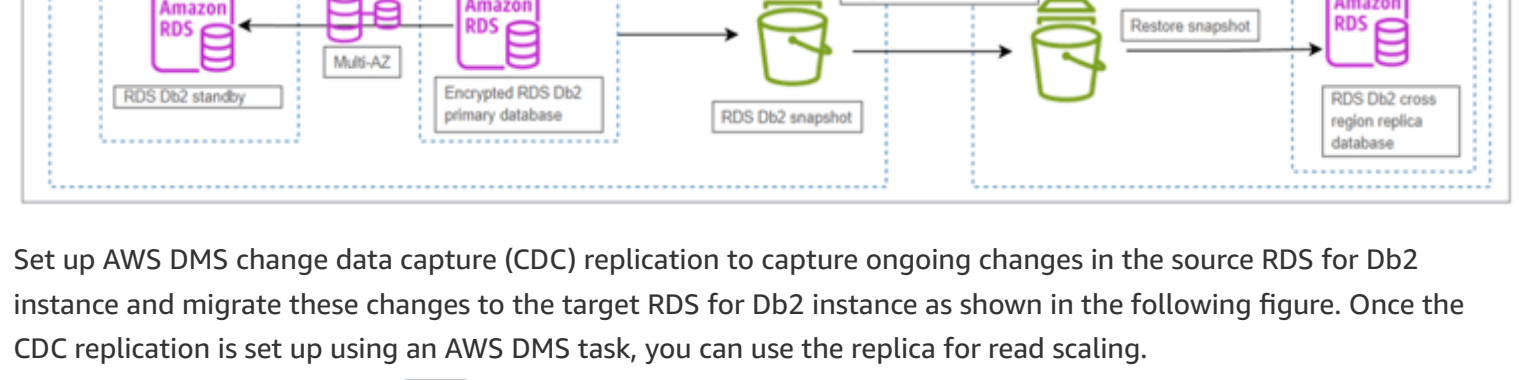
AWS DMS is a managed migration and replication service that helps move your database and analytics workloads to AWS quickly, securely, and with minimal downtime and data loss.

The solution involves three steps:

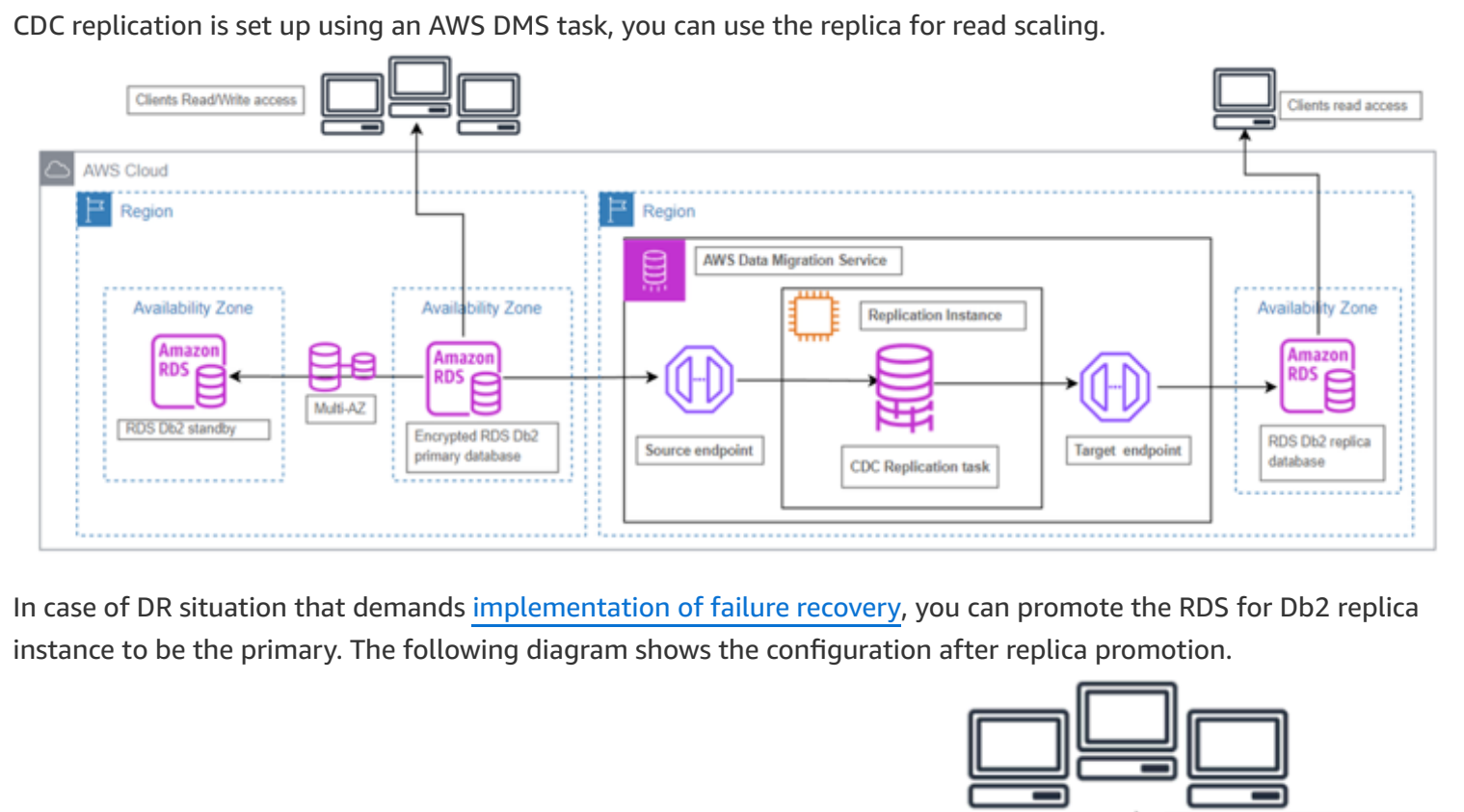
1. For a full load, perform a backup of the source RDS for Db2 DB instance and restore the backup in the desired destination AWS Region as shown in the following figure.



2. Set up AWS DMS change data capture (CDC) replication to capture ongoing changes in the source RDS for Db2 instance and migrate these changes to the target RDS for Db2 instance as shown in the following figure. Once the CDC replication is set up using an AWS DMS task, you can use the replica for read scaling.



3. In case of DR situation that demands implementation of failure recovery, you can promote the RDS for Db2 replica instance to be the primary. The following diagram shows the configuration after replica promotion.


Clients Read/Write access
AWS Cloud
Move workload to different Region
Region
Availability Zone
Amazon RDS
RDS Db2 standby
Multi-AZ
Availability Zone
Amazon RDS
Encrypted RDS Db2 Primary database
Region
Availability Zone
Amazon RDS
RDS Db2 Primary/Stand-alone database

We walk through the details of these steps in the following sections.

Prerequisites

To follow along with this post, you should have the following prerequisites:

- An AWS account
- An RDS for Db2 instance
  - Create a custom parameter and option group
    - In the parameter group set `db2_ats_enable` to `YES`. This is needed to propagate the sequences from the primary to the replica. This is a dynamically modifiable parameter and is required by the administrative task scheduler.
  - In order to maintain consistency across Regions, perform the following steps before taking the backup
    - Copy the parameter and option groups from the source to the target Regions
    - If you copy an encrypted snapshot across Regions, you must specify a KMS key valid in the destination AWS Region. It can be a Region-specific KMS key, or a multi-Region key. For more details refer to Handling encryption while copying DB snapshots.
- Familiarity with RDS Db2 Snapshot and AWS DMS
- All the AWS DMS Prerequisites completed on the source database
- All the following AWS DMS prerequisites on both the source and target databases
  - Prerequisites when using Db2 LUW as a source for AWS DMS
  - Using Amazon RDS for Db2 and IBM Db2 LUW as a target for AWS DMS
- Create the following objects in the source database to allow the sequences to be propagated to the replica
  - A table to keep track of the current value of sequences. Place this table in a schema that is being replicated through AWS DMS.

```
CREATE TABLE "SCHEMA_NAME"."SEQUENCE_TRACKER"
(
 "SEQSCHEMA"           varchar(128 octets) NOT NULL ,
 "SEQNAME"             varchar(128 octets) NOT NULL ,
 "NEXTCACHEFIRSTVALUE" decimal(31,0)
)
IN "USERSPACE1" organize BY ROW;

ALTER TABLE "SCHEMA_NAME"." SEQUENCE_TRACKER" DATA CAPTURE changes include longvar c

ALTER TABLE "SCHEMA_NAME"."SEQUENCE_TRACKER" ADD PRIMARY KEY ("SEQSCHEMA", "SEQNAME"

GRANT control ON TABLE "SCHEMA_NAME"."SEQUENCE_TRACKER" TO USER "ADMIN";
```

  - A stored procedure to track the sequences

```
CREATE PROCEDURE schema_name.track_sequence()
  SPECIFIC track_sequence
  LANGUAGE SQL
  BEGIN
    DECLARE stmt VARCHAR(100);
    SET stmt = 'DELETE FROM SCHEMA_NAME.SEQUENCE_TRACKER';
    EXECUTE IMMEDIATE stmt;
    SET stmt = 'INSERT INTO SCHEMA_NAME.SEQUENCE_TRACKER SELECT SEQSCHEMA,SEQNAME,NE
    EXECUTE IMMEDIATE stmt;
END;
```

  - A scheduled task to populate the table created in the preceding step

```
CALL SYSPROC.ADMIN_TASK_ADD
(
      'TRACK SEQUENCES EVERY 1 MINUTES',
       NULL,
       NULL,
       NULL,
      '*/1 * * * *',
      'SCHEMA_NAME',
      'TRACK_SEQUENCE',
       NULL,
       NULL,
       NULL
);
```

Solution walkthrough: Create self-managed replicas

In this section, we walk you through the detailed steps required to perform a full load and set up change data capture replication between the primary and replica RDS for Db2 instances using DMS for achieving disaster recovery and read scaling.

For this post, we provide the naming conventions listed in the following table for source and target RDS for Db2 databases for the sake of simplicity and better understanding.

	RDS for Db2 instance name	Region
Source	rds-db2-primary	us-east-1
Target	rds-db2-replica	us-east-2

Perform a backup of the source instance and restore the backup in the destination Region

In this section, we walk through the steps to perform a backup of the source AWS KMS encrypted RDS for Db2 DB instance (`rds-db2-primary`) in Region `us-east-1` and restore the backup in the destination Region `us-east-2`.

1. Increase log retention in the Source RDS for Db2 Instance `rds-db2-primary`
   Because we're using AWS DMS for CDC in this solution in the subsequent steps and when using Amazon RDS for Db2 LUW as a source, make sure that the archive logs are available to AWS DMS. Because AWS managed Db2 databases purge the archive logs as soon as possible, you should increase the length of time that the logs remain available.The default archive log retention is 0 hours, while the maximum retention is 168 hours (7 days). The recommended minimum retention is 24 hours.For example, to increase log retention to 24 hours, run the following command from the rdsadmin database.

```
db2 connect to rdsadmin user <master_username> using <master_password>
db2 "call rdsadmin.set_archive_log_retention( ?, '<DBNAME>', '24')"
```

2. Reboot the source RDS for Db2 instance
   It is essential to reboot the RDS for Db2 instance for the DMS CDC task to initiate from a specific timestamp.
   A. On the Amazon RDS console, navigate to the source (primary) DB instance. On the Actions menu, choose Reboot to reboot the instance.
   B. To reboot a DB instance using the AWS CLI, call the `reboot-db-instance` command. The following code snippet shows a simple reboot for Linux, macOS, or Unix

```
aws rds reboot-db-instance --db-instance-identifier rds-db2-primary
```

3. Verify the oldest open transactions in the source RDS for Db2 instanceData consistency is critical for a relational database management system (RDBMS), so that data remains accurate after transactions. At the time of creating an RDS for Db2 snapshot, there might be ongoing transactions in the database. Because we use an Amazon RDS for Db2 snapshot for a full load and AWS DMS CDC, it is essential to record the timestamp of the oldest open transaction start position or the current timestamp, whichever is older. This makes sure that AWS DMS replication stays in sync with the source database. Gaps or missing transactions between the snapshot and the start of replication could cause inconsistencies in the target database.To maintain data consistency and integrity, we recommend initiating the Amazon RDS for Db2 backup mentioned in the next step during off-peak hours when there are no open transactions.To fetch the timestamp (UTC time zone) of the oldest open transaction or the current timestamp, whichever is older in the source RDS for Db2 database use the following SQL query. This timestamp will be used as a starting point for AWS DMS CDC replication in subsequent steps.

```
WITH TEMP AS (
SELECT uow.uow_start_time-current timezone AS "CDC Start Time"
FROM   TABLE(Mon_get_connection(CAST(NULL AS BIGINT), -1)) AS con,
       TABLE(Mon_get_unit_of_work(NULL, -1)) AS uow
WHERE  con.application_handle = uow.application_handle
       AND uow.uow_log_space_used != 0
UNION
SELECT CURRENT timestamp - CURRENT timezone AS "CDC Start Time"
FROM SYSIBM.sysdummy1
)
SELECT min("CDC Start Time") FROM  TEMP;
```

4. To create an DB snapshot backup of the source RDS for Db2 instance follow the steps described in Creating a DB snapshot for a Single-AZ DB instance.
5. To copy the source Amazon RDS for Db2 snapshot to the target Region (for this post, `us-east-2`), follow the steps in Cross-Region snapshot copying.

   The following code shows the AWS CLI commands for Linux, macOS, or Unix to copy the snapshot to the target Region.

```
aws rds copy-db-snapshot \
--source-db-snapshot-identifier arn:aws:rds:us-east-1:<AWS Account>:snapshot:rds-db2-pr
--target-db-snapshot-identifier rds-db2-primary-snapshot-copy-targetregion \
--kms-key-id arn:aws:kms:us-east-2:<AWS Account>:key/mrk-503e6368fde54d2caebeb5 \
--region us-east-2
```

6. To restore the source Amazon RDS for Db2 snapshot to the target Region (`us-east-2`), follow the steps in Restoring to a DB instance.

   The following code shows the AWS CLI commands for Linux, macOS, or Unix to restore the snapshot into the target Region.

```
aws rds restore-db-instance-from-db-snapshot \
    --db-instance-identifier rds-db2-replica \
    --db-snapshot-identifier arn:aws:rds:us-east-2:<AWS Account>:snapshot:rds-db2-prima
    --db-instance-class db.m6i.12xlarge \
    --storage-type gp3 \
    --vpc-security-group-ids <sg group> \
    --db-subnet-group-name <subnet group> \
    --no-publicly-accessible \
    --no-auto-minor-version-upgrade \
    --port <port> \
    --db-parameter-group-name rds-db2-ohio \
    --no-multi-az \
    --region us-east-2
```

7. (Optional) Prewarm the target database to reduce the impact of lazy loading

Amazon RDS uses Amazon Elastic Block Store (Amazon EBS) as underlying storage. Amazon RDS snapshots (automated and manual) are saved in Amazon Simple Storage Service (Amazon S3). For RDS instances that are restored from snapshots, the instances are made available as soon as the needed infrastructure is provisioned. However, an ongoing process continues to copy the storage blocks from Amazon S3 to the EBS volume; this is called lazy loading or initialization. After the RDS instance is restored and made available, clients can connect to it and the RDS instance is open for both read and write activity. During the query or transaction processing, if a requested data block is already in Amazon EBS, there is no additional latency to fetch it. When a requested data block is not in Amazon EBS, it is immediately loaded from Amazon S3 into the EBS volume, which may cause additional I/O latency. Subsequent requests for previously accessed data blocks don't introduce extra I/O latency.

As a best practice we recommend prewarming the RDS for Db2 DB instance before starting CDC to reduce the impact of lazy loading. You can use the following methods to decrease the duration of lazy loading in the target RDS for Db2 instance.

- Run Db2 inspect
- Scanning Database tables
- RUNSTATS command on all tables and indices

Set up AWS DMS CDC replication to capture ongoing changes in the source instance and migrate these changes to the target instance

In this section, we walk through the steps to set up AWS DMS CDC replication to capture ongoing changes in the source RDS for Db2 instance (`rds-db2-primary`) and migrate these changes to the target (`rds-db2-replica`) RDS for Db2 instance. The following are best practices to be implemented in the replica database before starting CDC using AWS DMS:

- Understand AWS DMS limitations in the source and target databases, especially unsupported objects and data types
- Database activities that are not logged such as loads can't be replicated
- Database sequence value can't be replicated
- Drop triggers in the target database and enable them either before the cutover process or while activating the replica
  - Save the data definition language (DDL) of all objects with a command such as the following. For complete syntax of the db2look command refer to the IBM Documentation.

```
db2look -d <database_name> -i <remote_user_name> -w <remote_user_password> -z <schema
```

  - Use the following command to generate Drop Trigger DDL on the target database

```
db2 -x "SELECT 'DROP TRIGGER '|| rtrim(TRIGSCHEMA) || '.' || rtrim(TRIGNAME) ||';' F
```

- If you choose batch apply mode in the CDC task, you need to disable the constraints on the target and enable them either before the cutover process or while activating the replica
- Convert identity columns to `generate by default` if they are defined as `generate always`. These following scripts generate the alter table commands

```
db2 -x "SELECT 'ALTER TABLE ' || rtrim(TABSCHEMA) || '.' ||rtrim(TABNAME) || ' ALTER CO

db2 -x "SELECT 'ALTER TABLE ' || rtrim(TABSCHEMA) || '.' ||rtrim(TABNAME) || ' ALTER CO
```

- Take a snapshot of the target RDS for Db2 database so it can be used to restore the full snapshot in case of any issues with AWS DMS replication and before restarting AWS DMS replication

To create an AWS DMS CDC-only task to capture ongoing changes in the source instance and migrate these changes to the target instance, follow these steps.

To configure an AWS DMS replication, you need the following components:

- A source endpoint
- A target endpoint
- A replication instance
- An AWS DMS migration task

Refer to the AWS DMS User Guide or Migrate from self-managed Db2 to Amazon RDS for Db2 using AWS DMS to learn more details about these components and how to create them.

1. To create an AWS DMS CDC-only task, you need to set the start position of replication by configuring the source endpoint settings with extra connection attributes of `StartFromContext`. This provides data consistency and integrity and helps avoid gaps or missing transactions between the snapshot and the start of replication. Additionally, configure the DMS error handling task settings:
   - "ApplyErrorInsertPolicy": "INSERT_RECORD" to handle duplicate records and
   - "ApplyErrorUpdatePolicy": "UPDATE_RECORD" to handle missing records. This option disables LOB column support for the task.

   You can find additional information on these parameters at Error handling task settings.
2. Set the DMS CDC replication starting point as follows:
   To ensure data consistency, the starting point for DMS CDC replication should be the timestamp (in UTC time zone) of the oldest open transaction or the current timestamp (taken before initiating DB snapshot), whichever is older, in the source RDS for Db2 database that you captured in the preceding steps.
   For example: `StartFromContext=timestamp: 2024-08-31T15:20:09` The following screenshot shows the timestamp in the **Endpoint settings**.

For best practices on optimizing your AWS DMS task, refer to Performance optimization of full load and ongoing replication tasks from self-managed Db2 to Amazon RDS for Db2.log.

Use RDS for Db2 replica DB instance for Read scaling

Using RDS for Db2 replica DB instance can increase performance in scenarios involving heavy read operations, such as business reporting and read queries from applications. Instead of burdening your production DB instance with business reporting queries, you can route these queries to a replica. This approach helps manage and distribute read traffic more effectively. By doing so, you can elastically scale out beyond the capacity constraints of a single DB instance for read-heavy database workloads. Directing excess read traffic to one or more replicas enhances both performance and scalability for read-intensive workloads.

The steps to identify the endpoint for the RDS for Db2 replica instance and redirect read traffic to it are the following:

1. Finding the endpoint of your Amazon RDS for Db2 replica instance
   Each Amazon RDS for Db2 instance has an endpoint, and each endpoint has the DNS name and port number for the DB instance. To connect to your Amazon RDS for Db2 replica DB instance with a client application, you need the DNS name and port number for your DB instance. To find the endpoint refer to Finding the endpoint of your Amazon RDS for Db2 DB instance
2. Redirecting read traffic to the RDS for Db2 replica instance.
   Once you have the end point of the RDS for Db2 replica instance (`rds-db2-replica`), you can use this endpoint to connect to replica instance and redirect read traffic to it. The replica (`rds-db2-replica`) database allows both read and write access to users. Therefore, if you want to use this replica for read scaling, you need to implement logic at the application layer to direct only read traffic to the replica. Additionally, you have the option to create a new RDS for Db2 database user with read-only access on the replica instance. This user can be used in the application to connect to the replica and route only read traffic to it.
3. (Optional) Create a READONLY user in the target database and revoke all other connections to the target database to prevent updates

The replica (`rds-db2-replica`) database allows both read and write access to users. As best practice, you can generate scripts to revoke the CONNECT privilege from existing connections and re-grant it before promoting the replica (`rds-db2-replica`) database to the new primary database (read-write access) in the event of a disaster recovery scenario.

To connect to the replica database on the RDS for Db2 replica instance (`rds-db2-replica`) instance and generate "grant and revoke connect privilege" scripts, run the following script.

```
db2 connect to <replica database> user <master_username> using <master_password>
db2 -x "

-- Generate GRANT statements
SELECT 'GRANT CONNECT ON DATABASE TO GROUP ' || GRANTEE || ';' FROM SYSCAT.DBAUTH
WHERE GRANTOR NOT IN ('SYSIBM', 'RDSDB') AND CONNECTAUTH = 'N'   AND GRANTEETYPE = 'G';
SELECT 'GRANT CONNECT ON DATABASE TO USER ' || GRANTEE || ';' FROM SYSCAT.DBAUTH
WHERE GRANTOR NOT IN ('SYSIBM', 'RDSDB') AND CONNECTAUTH = 'N'   AND GRANTEETYPE = 'U';
SELECT 'GRANT CONNECT ON DATABASE TO ROLE ' || GRANTEE || ';' FROM SYSCAT.DBAUTH
WHERE GRANTOR NOT IN ('SYSIBM', 'RDSDB')  AND CONNECTAUTH = 'N'  AND GRANTEETYPE = 'R';
" > grant_connect_priv.sql

db2 -x "
-- Generate REVOKE statements
SELECT 'REVOKE CONNECT ON DATABASE FROM GROUP ' || GRANTEE || ';' FROM SYSCAT.DBAUTH
WHERE GRANTOR NOT IN ('SYSIBM', 'RDSDB') AND CONNECTAUTH = 'Y' AND GRANTEETYPE = 'G';

SELECT 'REVOKE CONNECT ON DATABASE FROM USER ' || GRANTEE || ';' FROM SYSCAT.DBAUTH
```

Promote the RDS for Db2 replica DB instance for disaster recovery

You can promote the RDS for Db2 replica instance to be the primary instance in case of a DR situation for implementing failure recovery. Alternatively, you can convert it to a standalone instance for testing purposes.

Before promoting the replica database (`rds-db2-replica`), complete the following steps:

1. Check AWS DMS task for any latency issues or Amazon DMS CloudWatch logs for monitoring AWS DMS for any replication errors , and address them as needed. For more information see Troubleshooting migration tasks in AWS Database Migration Service or Debugging Your AWS DMS Migrations: What to Do When Things Go Wrong (Part 1)
2. Recreate triggers in the replica database that were dropped earlier.
3. Convert identity columns back to `generate always` if they were changed earlier with `identity.change_to_always.ddl`
4. Reset the identity restart value for each identity column in the replica database `rds-db2-replica` with the following script:

```
db2 connect to target-db user <adminuser> using <password> > /dev/null

db2 -x "SELECT rtrim(TABSCHEMA) || ' ' ||rtrim(TABNAME) || ' ' ||  rtrim(COLNAME)  FROM
> alter_identity.sql

cat /tmp/working.$$|while read SCHEMA_NAME TABLE_NAME COLUMN_NAME
do
  db2 connect to target-db user <adminuser> using <password> > /dev/null
  db2 -x "SELECT 'ALTER TABLE ${SCHEMA_NAME}.${TABLE_NAME} ALTER COLUMN ${COLUMN_NAME}
     case when max(${COLUMN_NAME}) is null then '1'
     else (max(${COLUMN_NAME}) + 1)
     end
     || ';'
  FROM ${SCHEMA_NAME}.${TABLE_NAME} " >> alter_identity.sql
done
db2 -tvf alter_identity.sql
```

5. If the primary database `rds-db2-primary` is available, the following steps should be taken:
   a. Retrieve the maximum value for each sequence object in the primary database
   b. Use those maximum values to reset the restart value in the replica database `rds-db2-replica`
6. If the primary database, `rds-db2-primary` , is unavailable, the recent values of the sequences will be stored in the `SCHEMA_NAME.SEQUENCE_TRACKER` table. In this case:
   a. Identify the NEXTCACHEFIRSTVALUE column in the `SCHEMA_NAME.SEQUENCE_TRACKER` table
   b. This value will be a couple of minutes behind the actual maximum value in the primary database.
   c. To help prevent the generation of duplicate sequence numbers, reset the restart value in the replica database `rds-db2-replica` to the NEXTCACHEFIRSTVALUE from the SEQUENCE_TRACKER table, plus an additional value (that is, number) to ensure uniqueness.
7. Enable the constraints in the replica database that were disabled earlier.
8. Re-grant the connection (run `grant_connect_priv.sql`) if it was revoked earlier.

To promote the replica in a DR situation, follow these steps:

1. Complete the steps in the previous section for the replica database (`rds-db2-replica`) before promoting it
2. Direct database read/write traffic to the promoted DB instance.
3. Create a replacement replica with the promoted DB instance as its source using the steps mentioned above.

To promote the replica in a DR testing situation, follow these steps:

1. Stop the DMS replication Task
2. Complete the steps in previous section for the replica database (`rds-db2-replica`) before promoting it
3. Connect the DR Test application (read/write traffic) to the promoted DB instance for testing
4. Once the testing is done drop the replica database (`rds-db2-replica`)
5. Create a new replica with the original DB instance (`rds-db2-primary`) as its source using the steps mentioned in the preceding section

Conclusion

In this post, we discussed an approach to creating a self-managed cross-Region replica of an RDS for Db2 DB instance. Maintaining a cross-Region replica is a crucial aspect of a comprehensive disaster recovery plan, as it helps protect against workload disruptions in the primary database's region.

Additionally, by creating a self-managed, you can gain the benefits of read scaling. When you distribute read-heavy workloads across multiple database instances, you significantly enhance the overall read throughput and performance of your Db2 deployment. This is especially beneficial for applications that predominantly focus on read operations, such as reporting, analytics, or customer-facing interfaces.

If you have any questions or comments, post your thoughts in the comments section.

About the Authors

Javeed Mohammed is a Sr. Database Specialist Solutions Architect with Amazon Web Services. He works with the Amazon RDS team, focusing on commercial database engines like Oracle and Db2. He enjoys working with customers to help design, deploy, and optimize relational database workloads in the AWS Cloud.

Feng Cai is a Database Consultant at AWS. He has a wide background in providing database services to different clients. Feng is currently a member of the AWS World-wide Public-Sector team, providing homogeneous and heterogeneous database migration support to customers.

Dean Capps is a Database Consultant at AWS and has experience providing database support on legacy, open systems, and cloud platforms. Dean is currently a member of the AWS World-wide Public-Sector team, providing homogeneous and heterogeneous database migration support to public sector clients.

Ahmed Darwish is a Database Engineer in Database Migration Service at AWS specializing in Oracle and DB2. His expertise contributes to the advanced technical aspects of database migration and management within the extensive AWS services. He works with our customers to provide guidance and technical assistance on database migration projects, helping them improve the value of their solutions when using AWS.

Karthik Gopalakrishnan, is a Sr. Technical Product Manager, at AWS who leads the delivery of RDS for Db2 in collaboration with Engineering, GTM, and Sales and Marketing teams. He is passionate about transforming customers' needs and challenges into product solutions.

Like Share

Comments

Log in to comment

Learn
What Is AWS?
What Is Cloud Computing?
What Is Agentic AI?
Cloud Computing Concepts Hub
AWS Cloud Security
What's New
Blogs
Press Releases

Resources
Getting Started
Training
AWS Trust Center
AWS Solutions Library
Architecture Center
Product and Technical FAQs
Analyst Reports
AWS Partners

Developers
Builder Center
SDKs & Tools
.NET on AWS
Python on AWS
Java on AWS
PHP on AWS
JavaScript on AWS

Help
Contact Us
File a Support Ticket
AWS re:Post
Knowledge Center
AWS Support Overview
Get Expert Help
AWS Accessibility
Legal

Back to top ↑

Amazon is an Equal Opportunity Employer: Minority / Women / Disability / Veteran / Gender Identity / Sexual Orientation / Age.

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved.